Lab 5 Report

Name: Ruturaj Sushil Ghodke
Student No: 3775789

Performance Testing Table (with time in seconds)

N	**P=1**	**P=2**	**P=4**	**P=8**
1000	0.048682	0.024612	0.014378	0.011914
5000	0.320025	0.244413	0.416810	0.277130
10000	1.302046	1.019521	0.897778	0.862529
20000	19.205166	13.404228	9.656185	9.948556

Speedup Based on the above values

N	**P=1**	**P=2**	**P=4**	**P=8**
1000	1	1.97	3.38	4.08
5000	1	1.30	0.76	1.15
10000	1	1.27	1.45	1.50
20000	1	1.43	1.98	1.93

Attaching screenshot of the above lab experiment below:

```
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpicc -O3 lab5.c -o lab5
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 4 ./lab5 1000
N = 1000, Processes = 4, Time = 0.034634 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 1 ./lab5 1000
N = 1000, Processes = 1, Time = 0.048682 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 1 ./lab5 5000
N = 5000, Processes = 1, Time = 0.320025 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 1 ./lab5 10000
N = 10000, Processes = 1, Time = 1.302046 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 1 ./lab5 20000
N = 20000, Processes = 1, Time = 19.205166 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 2 ./lab5 1000
N = 1000, Processes = 2, Time = 0.024612 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 2 ./lab5 5000
N = 5000, Processes = 2, Time = 0.244413 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 2 ./lab5 10000
N = 10000, Processes = 2, Time = 1.019521 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 2 ./lab5 20000
N = 20000, Processes = 2, Time = 13.404228 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 4 ./lab5 1000
N = 1000, Processes = 4, Time = 0.014378 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 4 ./lab5 5000
N = 5000, Processes = 4, Time = 0.416810 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 4 ./lab5 10000
N = 10000, Processes = 4, Time = 0.897778 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 4 ./lab5 20000
N = 20000, Processes = 4, Time = 9.656185 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 8 ./lab5 1000
N = 1000, Processes = 8, Time = 0.011914 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 8 ./lab5 5000
N = 5000, Processes = 8, Time = 0.277130 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 8 ./lab5 10000
N = 10000, Processes = 8, Time = 0.862529 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$ mpirun -np 8 ./lab5 20000
N = 20000, Processes = 8, Time = 9.948556 seconds
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Lab 5$
```

Analysis of Benchmarking results:

1. **Small Problem Size (N=1000):** We see the best relative speedup here (4.08 at P=8), likely because the data fits entirely within the CPU cache, minimizing memory latency.
2. **The N=5000 Anomaly:** At P=4, the execution time spiked to 0.416 seconds, resulting in a speedup of only 0.76 (slower than serial). This could be attributed to temporary system noise, network congestion during the MPI_Scatter phase, or cache contention on my machine.
3. **Large Problem Size (N=20000):** There is a massive jump in the serial execution time (19.2s). While parallelization cuts this down to ~9.9s, the speedup stalls after P=4. This suggests we have hit a memory bandwidth bottleneck where the processors are waiting for data rather than performing calculations.

Discussion Questions:

1. **Does the program achieve linear speedup?**
   No, the program does not achieve linear speedup. For a linear speedup, P=8 should result in a speedup of approximately 8.0. Instead, the maximum observed

speedup was 4.08. As P increases, the gap between actual speedup and theoretical linear speedup widens.

2. **What factors limit scalability?**

   Several factors contribute to the sub-linear scaling observed in the data:

- **MPI Communication Overhead:** The time taken by MPI_Scatter, MPI_Bcast, and MPI_Gather to move data across the network increases as more processes are added.
- **Synchronization Costs:** The use of MPI_Barrier() ensures all processes start and stop together, but it means faster processes must idle while waiting for the slowest one.
- **Memory Bandwidth:** In matrix-vector multiplication, the processors must constantly fetch data from memory. If multiple processes on the same node compete for the same memory bus, performance saturates regardless of how many cores are used.

3. **Why does increasing N sometimes improve speedup?**

   Increasing N increases the **computation-to-communication ratio**. While the communication cost grows with N, the computational work (the nested loops) grows at a rate of N^2. With a larger N, the processors spend a higher percentage of their time doing math and a lower percentage of their time waiting for MPI calls to finish.

4. **Why can extremely large values of N cause crashes?**

   Dense matrices require significant contiguous memory. Since each double is 8 bytes, an N * N matrix requires approximately (8 * N * N) bytes.

a. For N=20,000, rank 0 must allocate approx 3.2 GB just for matrix A.
b. If N=1,000,000, the memory required would be 8 TB, which exceeds the RAM of most standard systems and leads to Segmentation faults or out-of-memory errors.